**UPTD BALAI LATIHAN KERJA (BLK) PROVINSI GORONTALO**

Pelatihan Berbasis Kompetensi

## JUNIOR WEB DEVELOPER

# MODUL KERJA 3

## Data Eksternal, Library CDN, dan Proyek Akhir

Bagian A: HTML, CSS, JavaScript

Batch #5 · Kelas: Junior Web Developer

Instruktur: Rahmat S.A.F. Badaru

2026

# Daftar Isi


- A. Identitas Modul
- B. Pemetaan Unit Kompetensi dan Kriteria Capaian
- C. Tujuan Pembelajaran
- D. Petunjuk Belajar
- Kegiatan Belajar 3.1: JSON dan `fetch()` (Hari 9, Pagi)
- Kegiatan Belajar 3.2: Library atau Komponen Pre-Existing lewat CDN (Hari 9, Siang)
- Kegiatan Belajar 3.3: Proyek Akhir Bagian A (Hari 10)
- E. Lembar Kerja Pendamping
- F. Lembar Asesmen Modul 3 dan Proyek Akhir Bagian A
- G. Glosarium
- H. Referensi
- Lampiran A: Kode Acuan Kopi Nusantara Versi 3
- Lampiran B: Panduan Presentasi Proyek Akhir

# A. Identitas Modul

| Komponen | Keterangan |
|---|---|
| Nama modul | Data Eksternal, Library CDN, dan Proyek Akhir |
| Bagian | A: HTML, CSS, JavaScript (Modul 3 dari 3, terakhir) |
| Durasi | 16 JP (1 JP = 45 menit), 2 hari × 8 JP, Hari 9-10 |
| Prasyarat | Modul 2 selesai: proyek Kopi Nusantara Versi 2 dan halaman bertema sendiri Versi 2 (data, render, filter, formulir) |
| Luaran | (1) Proyek latihan bersama "Kopi Nusantara Versi 3"; (2) **Proyek Akhir Bagian A**: halaman bertema sendiri lengkap dengan data JSON, minimal satu library CDN, README, dan presentasi singkat |
| Perangkat | Komputer atau laptop, koneksi internet minimal 5 Mbps |
| Perangkat lunak | Laragon, Visual Studio Code, browser dengan DevTools (khususnya tab Network) |

**Gambaran modul.** Pada Modul 2, data (`data.js`) dimuat lewat tag `<script>` dan sudah tersedia sejak awal. Pada Modul 3, data dipindahkan ke **berkas JSON terpisah** dan diambil dengan `fetch()`, meniru cara aplikasi web sungguhan mengambil data dari server. Modul ini juga memperkenalkan **library pre-existing** lewat CDN sebagai pembanding, sebelum peserta menyelesaikan **Proyek Akhir Bagian A**.

| Hari | Fokus | Luaran |
|---|---|---|
| 9 (pagi) | JSON dan `fetch()`, `async/await` dasar | Kopi Nusantara Versi 3 |
| 9 (siang) | Library CDN sebagai pembanding | Latihan library kecil |
| 10 | Proyek Akhir Bagian A dan asesmen | Proyek Akhir dan presentasi |

**Catatan penting.** Bagian ini adalah **fondasi**, bukan pendalaman. Konsep `async/await` diajarkan sampai tingkat "peserta bisa memuat dan menampilkan data JSON", bukan sampai menguasai *event loop* atau *race condition* secara mendalam. Pendalaman itu tersedia sebagai pengayaan opsional. Library CDN diajarkan **sebagai pembanding saja**; instalasi lewat Composer dan sejenisnya menjadi bagian Bagian B (PHP), bukan modul ini.

# B. Pemetaan Unit Kompetensi dan Kriteria Capaian

Modul ini menuntaskan enam Unit Kompetensi (UK) untuk Bagian A. UK 1.6 (library pre-existing) mendapat porsi utamanya di sini.

| UK | Kode Unit | Nama Unit Kompetensi | JP di Modul 3 |
|---|---|---|---|
| 1.1 | J.620100.005.02 | Mengimplementasikan user interface | 4 |
| 1.2 | J.620100.010.01 | Menerapkan perintah eksekusi bahasa pemrograman | 1 |
| 1.3 | J.620100.015.01 | Menyusun fungsi, file, atau sumber daya pemrograman dalam organisasi yang rapi | 3 |
| 1.4 | J.620100.016.01 | Menulis kode dengan prinsip sesuai guidelines dan best practices | 1 |
| 1.5 | J.620100.017.02 | Mengimplementasikan pemrograman terstruktur | 3 |
| 1.6 | J.620100.019.02 | Menggunakan library atau komponen pre-existing | 4 |

| UK | Kode Unit | Nama Unit Kompetensi | JP di Modul 3 |
|---|---|---|---|
| | | **Jumlah** | **16** |

*Dengan Modul 3 selesai, seluruh alokasi Bagian A tercapai: 30 + 10 + 8 + 6 + 22 + 4 = **80 JP** dari 6 UK.*

| Kode | Kriteria capaian | Dilatihkan di |
|---|---|---|
| 1.1-K3 | Mengimplementasikan rancangan UI menjadi halaman yang lengkap dan berfungsi | KB 3.3 |
| 1.2-K2 | Source code dieksekusi sesuai mekanisme (server lokal, bukan `file://`) | KB 3.1 |
| 1.2-K3 | Hasil eksekusi diidentifikasi sesuai skenario (status loading, sukses, error di tab Network) | KB 3.1 |
| 1.3-K1 | Nama berkas dan folder data (`data/produk.json`) sesuai konteks | KB 3.1, 3.3 |
| 1.3-K2 | Source code dilengkapi komentar sesuai konteks | KB 3.1, 3.3 |
| 1.3-K4 | `README.md` mencerminkan struktur folder dan cara menjalankan proyek terkini | KB 3.3 |
| 1.4-K1 | Source code ditulis sesuai guidelines (penanganan error yang rapi, tidak ada kode mati) | KB 3.1, 3.3 |
| 1.5-K5 | Membuat program yang mengakses data dari media penyimpanan (berkas JSON) | KB 3.1 |
| 1.5-K6 | Memperbaiki permasalahan pada program (galat jaringan, JSON tidak valid) | KB 3.1 |
| 1.6-K1 | Memilih library yang sesuai kebutuhan | KB 3.2 |
| 1.6-K2 | Memasang atau menghubungkan library (lewat CDN) | KB 3.2 |
| 1.6-K3 | Memahami lisensi, hak cipta, dan pembaruan versi library | KB 3.2 |

## C. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, Anda mampu:

1. Menjelaskan mengapa data pada aplikasi web modern sering diambil dari berkas atau server, bukan ditulis langsung di kode.
2. Menulis dan membaca berkas JSON yang valid.
3. Memuat data dengan `fetch()` dan `async/await`, serta menangani kondisi loading, sukses, dan error.
4. Menjelaskan mengapa proyek berbasis `fetch()` harus dijalankan lewat server lokal.
5. Memilih, memasang, dan memakai satu library pre-existing lewat CDN sebagai pembanding penulisan manual.
6. Menjelaskan dasar lisensi dan pentingnya memperbarui library.
7. Menyelesaikan **Proyek Akhir Bagian A**: halaman bertema sendiri yang lengkap, terdokumentasi, dan siap dipresentasikan.

## D. Petunjuk Belajar

- Modul ini lebih singkat dari Modul 1 dan 2 (16 JP), sehingga **kecepatan penting**. Ikuti setiap langkah praktik tanpa menunda.
- **Jalankan proyek lewat server lokal** (Laragon sudah aktif) sepanjang modul ini. Alamat `file://...` akan gagal pada KB 3.1.
- Buka DevTools (`F12`), tab **Network**, sejak awal KB 3.1. Tab ini menjadi alat utama pada modul ini.
- Hari 10 sepenuhnya untuk **Proyek Akhir**. Siapkan tema, data, dan mockup Anda (dari Modul 1) sebelum Hari 10 dimulai.

- Kode acuan lengkap ada di **Lampiran A**.

# Kegiatan Belajar 3.1: JSON dan `fetch()`

**Hari 9, Pagi · 4 JP · UK 1.5 (utama), 1.2, 1.3, dan 1.4**

**Tujuan KB 3.1.** Setelah mengikuti KB ini, Anda mampu menulis berkas JSON yang valid, memuat data dengan `fetch()` dan `async/await`, serta menampilkan status loading, sukses, dan error.

| Kegiatan | JP |
|---|---|
| Mengapa data dipisah ke berkas terpisah; dasar JSON | 1 |
| `fetch()`, Promise, dan `async/await` | 1 |
| Praktik: memindahkan `data.js` ke `data/produk.json` | 1 |
| Status loading dan error; tab Network | 1 |
| **Jumlah** | **4** |

## Materi 1: Mengapa Data Dipisah ke Berkas Terpisah

Pada Modul 2, `data.js` dimuat lewat `<script>` dan sudah tersedia sejak halaman dibuka. Ini sederhana, tetapi punya keterbatasan: setiap perubahan data mengharuskan Anda mengedit kode JavaScript, dan data tidak dapat diambil dari sumber lain (server, API pihak ketiga).

Aplikasi web sungguhan biasanya memuat data dari **server** saat halaman dibuka, sehingga:

- Data dapat diperbarui tanpa mengubah kode halaman.
- Data dapat dibagikan ke banyak aplikasi (web, mobile) dari satu sumber.
- Data dapat sangat besar dan tidak perlu dikirim seluruhnya sekaligus (topik lanjutan).

Langkah pertama menuju pola ini: pisahkan data ke berkas **JSON**, lalu ambil dengan `fetch()`.

## Materi 2: Dasar JSON

**JSON** (*JavaScript Object Notation*) adalah format teks untuk menyimpan data terstruktur, mirip objek dan array JavaScript, tetapi dengan aturan lebih ketat.

```
{
  "site": { "nama": "Kopi Nusantara", "email": "halo@kopinusantara.test" },
  "menu": [
    { "id": "beranda", "label": "Beranda" }
  ],
  "produk": [
    { "slug": "toraja", "nama": "Kopi Toraja", "harga": 85000 }
  ]
}
```

**Perbedaan JSON dengan objek JavaScript biasa:**

| Aturan | JSON | Objek JavaScript |
|---|---|---|
| Nama properti (key) | **Wajib** tanda kutip ganda: `"nama"` | Boleh tanpa kutip: `nama` |
| Tanda kutip nilai teks | **Wajib** kutip ganda `"..."` | Kutip ganda atau tunggal |
| Komentar | **Tidak diperbolehkan** | Diperbolehkan (`//`, `/* */`) |

| Aturan | JSON | Objek JavaScript |
|---|---|---|
| Koma di elemen/properti terakhir | **Tidak diperbolehkan** | Diperbolehkan |
| Fungsi di dalam nilai | **Tidak diperbolehkan** | Diperbolehkan |

**Kesalahan JSON paling umum:** koma berlebih di akhir array atau objek, tanda kutip tunggal, dan lupa tanda kutip pada nama properti. Kesalahan ini membuat **seluruh berkas gagal dibaca**, bukan hanya bagian yang salah.

## Materi 3: `fetch()`, Promise, dan `async/await`

Mengambil data lewat jaringan **butuh waktu**, sehingga tidak bisa selesai seketika seperti membaca variabel biasa. JavaScript menangani ini dengan konsep **asynchronous** (tidak menunggu di tempat).

`fetch(url)` mengembalikan sebuah **Promise**, yaitu "janji" bahwa hasilnya akan tersedia nanti. Untuk menunggu hasilnya, gunakan `await` di dalam fungsi yang ditandai `async`:

```
async function muatData() {
  const respons = await fetch("data/produk.json");
  const data = await respons.json();  // mengubah teks JSON menjadi objek JavaScript
  return data;
}
```

| Bagian | Fungsi |
|---|---|
| `async function` | Menandai bahwa fungsi ini boleh memakai `await` di dalamnya |
| `await fetch(url)` | Menunggu server membalas permintaan |
| `respons.json()` | Mengubah isi balasan (teks JSON) menjadi objek JavaScript; juga butuh `await` |

**Peringatan penting: `fetch()` tidak otomatis gagal pada error HTTP** (seperti 404, berkas tidak ditemukan). Ia hanya gagal pada masalah jaringan (misalnya server mati). Karena itu, status harus **diperiksa manual**:

```
async function muatData() {
  const respons = await fetch("data/produk.json");

  if (!respons.ok) {
    throw new Error("Gagal memuat data (HTTP " + respons.status + ")");
  }

  return await respons.json();
}
```

`respons.ok` bernilai `true` jika status antara 200 sampai 299 (sukses). `throw new Error(...)` menghentikan fungsi dan mengirim pesan kesalahan ke pemanggilnya.

**Menangkap error dengan `try` dan `catch`:**

```
async function mulai() {
  try {
    const data = await muatData();
    console.log("Data berhasil dimuat:", data);
  } catch (err) {
    console.error("Terjadi masalah:", err.message);
  }
}

mulai();
```

Kode di dalam `try` dijalankan seperti biasa. Jika terjadi `throw` di mana pun di dalamnya (termasuk di dalam fungsi yang dipanggil), eksekusi **langsung melompat** ke `catch`.

### Praktik 1: Memindahkan Data ke JSON

**Langkah 1.** Buat folder `data` di proyek Kopi Nusantara, lalu buat berkas `data/produk.json` berisi data yang sama seperti `js/data.js` Anda, ditulis ulang dalam format JSON:

```
{
  "site": {
    "nama": "Kopi Nusantara",
    "email": "halo@kopinusantara.test"
  },
  "menu": [
    { "id": "beranda", "label": "Beranda" },
    { "id": "produk", "label": "Produk" },
    { "id": "kontak", "label": "Kontak" }
  ],
  "produk": [
    { "slug": "toraja", "nama": "Kopi Toraja", "asal": "Sulawesi Selatan",
      "kategori": "arabika", "harga": 85000,
      "deskripsi": "Body tebal, aroma rempah dan cokelat gelap." },
    { "slug": "gayo", "nama": "Kopi Gayo", "asal": "Aceh",
      "kategori": "arabika", "harga": 80000,
      "deskripsi": "Rasa bersih dengan sentuhan karamel." },
    { "slug": "kintamani", "nama": "Kopi Kintamani", "asal": "Bali",
      "kategori": "arabika", "harga": 90000,
      "deskripsi": "Keasaman cerah seperti jeruk, ringan dan segar." },
    { "slug": "bajawa", "nama": "Kopi Bajawa", "asal": "Flores",
      "kategori": "arabika", "harga": 88000,
      "deskripsi": "Aroma floral dengan aftertaste manis." },
    { "slug": "lampung", "nama": "Robusta Lampung", "asal": "Lampung",
      "kategori": "robusta", "harga": 60000,
      "deskripsi": "Pahit tegas dan kental, cocok untuk kopi susu." },
    { "slug": "temanggung", "nama": "Robusta Temanggung", "asal": "Jawa Tengah",
      "kategori": "robusta", "harga": 65000,
      "deskripsi": "Rasa earthy dengan aroma kacang panggang." }
  ]
}
```

**Langkah 2.** Hapus (atau ganti nama menjadi `data.js.lama`) berkas `js/data.js`. Hapus juga baris `<script src="js/data.js"></script>` di `index.html`, karena data tidak lagi dimuat lewat cara itu.

**Langkah 3.** Pada bagian **paling atas** `js/app.js`, ganti deklarasi data:

```
// Sebelumnya: SITE, MENU, PRODUCTS berasal dari data.js (sudah tersedia sejak awal)
// Sekarang: diisi setelah fetch() berhasil
let SITE, MENU, PRODUCTS;
```

**Langkah 4.** Tambahkan fungsi pemuat data tepat di bawahnya:

```
async function muatData() {
  const respons = await fetch("data/produk.json");

  if (!respons.ok) {
    throw new Error("Gagal memuat data (HTTP " + respons.status + ")");
  }

  return await respons.json();
}

async function mulai() {
  const app = document.querySelector(".site-content");
  app.innerHTML = '<p class="status" role="status">Memuat data...</p>';

  try {
    const data = await muatData();
    SITE = data.site;
    MENU = data.menu;
    PRODUCTS = data.produk;

    // Kembalikan konten asli, lalu panggil fungsi render seperti Modul 2
    // (langkah selengkapnya pada Praktik 2)
  } catch (err) {
    app.innerHTML =
      '<div class="status error" role="alert"><strong>Data tidak dapat
dimuat.</strong><p>' +
      esc(err.message) + '</p></div>';
    console.error(err);
  }
}
```

**Langkah 5.** Ganti baris pemanggilan langsung di bagian inisialisasi (dahulu memanggil `renderMenu()` dan seterusnya secara langsung) menjadi:

```
mulai();
```

Bagian tampilan status loading dan error akan disempurnakan pada Materi 4. Untuk sekarang, uji dulu bahwa `console.log(data)` di dalam `mulai()` menampilkan objek data yang benar di Console.

## Materi 4: Status Loading, Error, dan Tab Network

Halaman yang memuat data dari luar perlu menangani **tiga kondisi**:

| Kondisi | Tampilan yang disarankan |
|---|---|
| Loading | Pesan “Memuat data…” (mencegah halaman terlihat kosong/rusak) |
| Sukses | Tampilan normal (menu, kartu, dan seterusnya) |
| Error | Pesan yang jelas, tanpa istilah teknis yang membingungkan pengguna |

**Tab Network di DevTools.** Buka `F12`, tab **Network**, lalu segarkan halaman. Anda akan melihat permintaan ke `produk.json` beserta **status** (200 untuk sukses, 404 untuk tidak ditemukan), **waktu**, dan **isi balasan**. Ini adalah alat utama untuk memeriksa masalah pemuatan data.

### Praktik 2: Menyelesaikan Alur Loading

Lengkapi `mulai()` agar memanggil ulang fungsi render Modul 2 setelah data siap:

```
async function mulai() {
  const app = document.querySelector(".site-content");
  const isiAsli = app.innerHTML;   // simpan kerangka asli sebelum diganti pesan
loading

  app.innerHTML = '<p class="status" role="status">Memuat data...</p>';

  try {
    const data = await muatData();
    SITE = data.site;
    MENU = data.menu;
    PRODUCTS = data.produk;

    app.innerHTML = isiAsli;   // kembalikan kerangka asli (section beranda, produk,
kontak)

    // Ambil ulang elemen setelah kerangka dikembalikan
    inisialisasiElemen();
    document.getElementById("tahun").textContent = new Date().getFullYear();
    renderMenu();
    renderFilter();
    renderProduk();
    pasangEventFormulir();
  } catch (err) {
    app.innerHTML =
      '<div class="status error" role="alert"><strong>Data tidak dapat
dimuat.</strong><p>' +
      esc(err.message) +
      '</p><p>Pastikan halaman dibuka lewat server lokal (http://), bukan langsung dari
berkas.</p></div>';
    console.error(err);
  }
}

mulai();
```

Karena elemen seperti `menuEl` dan `formKontak` sebelumnya diambil **sekali di awal berkas**, sedangkan sekarang kerangka sempat diganti pesan loading, elemen itu perlu diambil ulang. Bungkus pengambilan elemen dan pemasangan event formulir ke dalam dua fungsi kecil:

```
let menuEl, filterEl, jumlahEl, daftarEl, formKontak, statusEl;

function inisialisasiElemen() {
  menuEl = document.getElementById("menu");
  filterEl = document.getElementById("filter-produk");
  jumlahEl = document.getElementById("jumlah-produk");
  daftarEl = document.getElementById("daftar-produk");
  formKontak = document.getElementById("form-kontak");
  statusEl = document.getElementById("form-status");
}
```

```
function pasangEventFormulir() {
  formKontak.addEventListener("submit", function (e) {
    e.preventDefault();
    const nama = formKontak.nama.value.trim();
    const pesan = formKontak.pesan.value.trim();

    if (nama === "" || pesan === "") {
      statusEl.textContent = "Nama dan pesan wajib diisi.";
      return;
    }
    statusEl.textContent = "Terima kasih, " + nama + "! Pesan Anda sudah kami terima.";
    formKontak.reset();
  });
}
```

Pindahkan juga `addEventListener` untuk `filterEl` (klik tombol filter) ke dalam `inisialisasiElemen()` atau fungsi terpisah yang dipanggil sekali saja setelah elemen diambil ulang.

**Uji tiga kondisi:**

1. **Sukses:** buka halaman lewat Laragon. Menu dan kartu produk tampil seperti Modul 2.
2. **Error 404:** ubah sementara nama berkas menjadi `data/produk.jsn` (salah), segarkan halaman, dan amati pesan error serta status di tab Network. Kembalikan nama berkas setelah selesai menguji.
3. **Dibuka tanpa server:** klik dua kali `index.html` langsung dari folder (tanpa lewat Laragon). Amati pesan error di halaman dan di Console. Kembalikan ke alamat `http://localhost/...` setelah menguji.

## Latihan 1

1. Tambahkan satu produk baru **hanya dengan mengubah** `data/produk.json`, tanpa menyentuh `app.js`. Pastikan tidak ada kesalahan format JSON (gunakan validator jika perlu, lihat bagian H).
2. Hapus tanda kutip pada salah satu nama properti di JSON (misalnya `slug` menjadi `slug` tanpa kutip), simpan, lalu amati pesan error yang muncul. Kembalikan setelah diamati.
3. Jelaskan dengan kalimat sendiri: mengapa `fetch()` yang gagal pada 404 **tidak langsung** membuat blok `catch` berjalan tanpa pengecekan `respons.ok`?
4. *Tantangan:* tambahkan simulasi jeda dengan `await new Promise(r => setTimeout(r, 1000));` sebelum `fetch()` di `muatData()`, agar status “Memuat data…” sempat terlihat. Hapus lagi setelah diamati.

## Rangkuman KB 3.1

- Data dipisah ke berkas JSON agar mudah diperbarui dan mengikuti pola aplikasi web sungguhan.
- JSON mirip objek JavaScript, tetapi tanda kutip ganda wajib, tanpa komentar, dan tanpa koma berlebih.
- `fetch()` mengembalikan Promise; `await` di dalam fungsi `async` menunggu hasilnya.
- `fetch()` tidak otomatis gagal pada error HTTP; `respons.ok` harus diperiksa manual.
- Halaman perlu menangani tiga kondisi: loading, sukses, dan error. Tab Network membantu memeriksa semuanya.

## Cek Kemampuan KB 3.1

- ☐ Saya dapat menjelaskan perbedaan JSON dan objek JavaScript biasa.
- ☐ Saya dapat menjelaskan fungsi `async`, `await`, dan `fetch()` dengan kalimat sendiri.

- ☐ Proyek saya berhasil memuat data dari `data/produk.json`.
- ☐ Saya sudah menguji ketiga kondisi: sukses, error 404, dan dibuka tanpa server.
- ☐ Saya dapat membuka tab Network dan menjelaskan status permintaan yang terlihat di sana.

**Kesalahan umum:** lupa `await` (variabel berisi Promise, bukan datanya); lupa menandai fungsi dengan `async`; JSON memakai tanda kutip tunggal atau ada koma berlebih di akhir; lupa memeriksa `respons.ok`; halaman diuji dengan mengklik dua kali `index.html` alih-alih lewat Laragon.

# Kegiatan Belajar 3.2: Library atau Komponen Pre-Existing lewat CDN

**Hari 9, Siang · 4 JP · UK 1.6 (utama)**

**Tujuan KB 3.2.** Setelah mengikuti KB ini, Anda mampu menjelaskan apa itu library pre-existing, memilih library yang sesuai kebutuhan, memasangnya lewat CDN, serta memahami dasar lisensi dan pembaruan versi.

| Kegiatan | JP |
|---|---|
| Pengertian dan jenis library atau komponen pre-existing | 1 |
| Lisensi, hak cipta, dan versi | 1 |
| Memasang dan memakai library lewat CDN | 1 |
| Praktik: menambahkan satu library pada halaman latihan | 1 |
| **Jumlah** | **4** |

## Materi 1: Pengertian Library Pre-Existing

**Library** (pustaka) adalah kumpulan kode siap pakai yang ditulis orang lain untuk menyelesaikan masalah umum, sehingga Anda tidak perlu menulisnya dari nol. Contoh masalah umum: menampilkan ikon, memformat tanggal, atau membuat grafik.

**Keuntungan memakai library:**

- Menghemat waktu; masalah umum sudah diselesaikan dan diuji banyak orang.
- Biasanya lebih teruji dibanding kode buatan sendiri yang terburu-buru.

**Risiko yang perlu diperhatikan:**

- Menambah ukuran halaman (waktu muat lebih lama).
- Bergantung pada pihak lain: jika layanan CDN bermasalah, fitur ikut bermasalah.
- Tidak semua library bebas dipakai untuk segala keperluan (lihat Materi 2).

**Cara memasang library dalam pelatihan ini: CDN** (*Content Delivery Network*). Anda cukup menambahkan satu baris `<script>` atau `<link>` yang menunjuk ke alamat library di internet, tanpa mengunduh atau menginstal apa pun ke komputer.

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/.../nama-library.css">
<script src="https://cdn.jsdelivr.net/.../nama-library.js"></script>
```

**Catatan penting:** cara memasang library dengan **Composer** (pengelola paket untuk PHP) akan dipelajari di Bagian B. CDN dipilih di modul ini karena paling sederhana untuk proyek HTML, CSS, dan JavaScript murni, dan cukup untuk keperluan **pembanding**. Jangan mengubah struktur Proyek Induk (Kopi Nusantara) Versi 1-3; library CDN dicoba di **halaman latihan terpisah**.

## Materi 2: Lisensi, Hak Cipta, dan Versi

Setiap library punya **lisensi**, yaitu aturan penggunaan dari pembuatnya. Sebelum memakai library, periksa lisensinya.

| Jenis lisensi (contoh umum) | Artinya secara garis besar |
|---|---|
| MIT, Apache 2.0 | Bebas dipakai, diubah, bahkan untuk proyek komersial, dengan syarat ringan (biasanya menyertakan pemberitahuan hak cipta) |
| GPL | Bebas dipakai, tetapi proyek yang memakainya sering harus ikut dibuka sumber kodenya |
| Berbayar / proprietary | Perlu izin atau lisensi berbayar untuk penggunaan tertentu |

**Praktik baik:** periksa berkas `LICENSE` di halaman resmi library sebelum memakainya untuk proyek yang akan dipublikasikan atau dijual. Untuk latihan di pelatihan ini, library populer berlisensi MIT atau Apache 2.0 sudah cukup aman.

**Versi library.** Library terus diperbarui. Alamat CDN biasanya mencantumkan nomor versi, misalnya `.../4.6.0/....js`. Memakai nomor versi yang **tetap** (bukan "versi terbaru otomatis") membuat proyek Anda tidak tiba-tiba rusak ketika library diperbarui pembuatnya. Memperbarui versi berarti mengganti nomor tersebut **dengan sengaja**, lalu **menguji ulang** halaman Anda.

## Praktik 1: Memasang Satu Library CDN

Anda akan mencoba **Font Awesome** (ikon) pada halaman **latihan terpisah**, bukan pada Proyek Induk.

**Langkah 1.** Buat `latihan/cdn.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Latihan Library CDN</title>
  <link rel="stylesheet"
        href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.5.2/css/all.min.css">
  <style>
    body { font-family: Arial, sans-serif; padding: 2rem; }
    .kartu { display: flex; align-items: center; gap: 0.5rem; }
  </style>
</head>
<body>
  <h1>Latihan Library CDN: Font Awesome</h1>

  <div class="kartu">
    <i class="fa-solid fa-mug-hot"></i>
    <span>Kopi Toraja</span>
  </div>

  <div class="kartu">
    <i class="fa-solid fa-envelope"></i>
    <span>Hubungi Kami</span>
  </div>
</body>
</html>
```

**Langkah 2.** Buka di browser (boleh langsung dari berkas, karena ini hanya CSS dari CDN, bukan `fetch()`). Anda akan melihat ikon cangkir dan amplop.

**Langkah 3.** Buka tab **Network**, segarkan halaman, dan temukan permintaan ke `cdnjs.cloudflare.com`. Ini membuktikan berkas library benar-benar diambil dari internet, bukan dari komputer Anda. Jika internet mati, ikon tidak akan tampil.

**Langkah 4.** Bandingkan dengan cara **tanpa library**: seberapa banyak kode yang perlu Anda tulis sendiri untuk menggambar ikon cangkir kopi dengan SVG murni? Diskusikan dengan instruktur.

**Langkah 5 (opsional, sebagai pembanding lain).** Coba satu library tambahan sesuai arahan instruktur, misalnya Google Fonts (untuk jenis huruf) melalui:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Poppins&display=swap"
rel="stylesheet">
```

lalu pakai di CSS: `font-family: "Poppins", Arial, sans-serif;`.

## Latihan 2

1. Sebutkan dua library yang **berbeda jenis** dari Font Awesome (misalnya untuk grafik atau animasi) dan jelaskan kegunaannya (boleh dari pencarian sendiri, dengan bimbingan instruktur).
2. Buka halaman resmi Font Awesome, temukan informasi lisensinya, dan simpulkan: bolehkah dipakai untuk proyek tugas akhir Anda?
3. Jelaskan apa yang terjadi pada halaman `cdn.html` Anda jika koneksi internet terputus. Mengapa ini menjadi salah satu risiko yang disebutkan di Materi 1?
4. *Diskusi kelas:* kapan sebaiknya menulis kode sendiri, dan kapan sebaiknya memakai library?

## Rangkuman KB 3.2

- Library pre-existing menghemat waktu, tetapi menambah ketergantungan pada pihak lain.
- CDN adalah cara paling sederhana memasang library: cukup satu baris `<link>` atau `<script>`.
- Setiap library punya lisensi; periksa sebelum dipakai untuk proyek yang dipublikasikan.
- Nomor versi pada alamat CDN sebaiknya tetap, dan diperbarui secara sengaja lalu diuji ulang.
- Composer (untuk PHP) dipelajari di Bagian B; modul ini hanya membahas CDN.

## Cek Kemampuan KB 3.2

- ☐ Saya dapat menjelaskan keuntungan dan risiko memakai library.
- ☐ Saya berhasil memasang satu library lewat CDN dan melihatnya di tab Network.
- ☐ Saya dapat menjelaskan dasar lisensi MIT secara sederhana.
- ☐ Saya memahami mengapa nomor versi pada alamat CDN sebaiknya tetap.

**Kesalahan umum:** alamat CDN salah ketik (ikon atau gaya tidak tampil, tanpa pesan error yang jelas di halaman, hanya di tab Network/Console); lupa bahwa CDN butuh internet; memasang terlalu banyak library untuk kebutuhan yang sederhana.